Goldman Sachs

Python Project

Task 1: Data Cleaning and Formatting

● I have loaded the files in Jupiter Notebook editor in .CVS format.

● Importing all the necessary libraries

● I clean the field by stripping out symbols like currency signs or commas, convert it to a number, and deal with any non-numeric values by nulling or flagging them based on the rules.

- I check that the dates follow the expected format, correct or standardize them if needed, remove any invalid values.
- I ensure account types and transaction categories are standardized.

[1]:
```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt # task 4
from scipy.stats import zscore #task 4
from scipy import stats #task 4
import matplotlib.pyplot as plt #tash 5
from scipy.stats import ttest_ind  #task 6
```

[3]:
```python
GS = pd.read_csv(r"C:\Users\TRIDEV PAL\Documents\PYTHON HAND ON\goldman_sachs.csv")
```

[4]:
```python
GS.head(5)
```

[4]:

	TransactionID	CustomerID	AccountID	AccountType	TransactionType	Product	Firm	Region	Manager	TransactionDate	TransactionAmount	AccountBalance	Risk
0	33	CUST6549	ACC12334	Credit	Withdrawal	Savings Account	Firm C	Central	Manager 1	21-10-2023	87480.05448	74008.43310	0.7
1	177	CUST2942	ACC52650	Credit	Withdrawal	Home Loan	Firm A	East	Manager 3	20-06-2023	20315.74505	22715.83590	0.4
2	178	CUST6776	ACC45101	Current	Deposit	Personal Loan	Firm C	South	Manager 3	02-01-2023	10484.57165	42706.09210	0.6
3	173	CUST2539	ACC88252	Current	Withdrawal	Mutual Fund	Firm A	Central	Manager 2	25-07-2023	45122.27373	114176.56870	0.7
4	67	CUST2626	ACC21878	Savings	Withdrawal	Home Loan	Firm C	Central	Manager 4	25-07-2023	42360.79878	17863.02644	0.2

TASK 1

Remove/treat any special characters or non-numeric entries from financial fields. ¶

[5]:
```python
GS['TransactionAmount'] = (
    GS['TransactionAmount']
    .astype(str)
    .str.replace(r'[^0-9\.-]', '', regex=True)
    .astype(float)
)

GS['AccountBalance'] = (
    GS['AccountBalance']
    .astype(str)
    .str.replace(r'[^0-9\.-]', '', regex=True)
    .astype(float)
)
```

[6]:
```python
GS['AccountBalance'].head(10)
```

[6]:
```
0     74008.43310
1     22715.83590
2     42706.09210
3    114176.56870
4     17863.02644
5     61992.54273
6     92956.90293
7    140369.89900
8     78966.75783
9     69716.54246
Name: AccountBalance, dtype: float64
```

[7]:
```python
GS.dtypes
```

[7]:
```
TransactionID          int64
CustomerID            object
AccountID             object
AccountType           object
TransactionType       object
Product               object
Firm                  object
Region                object
Manager               object
TransactionDate       object
TransactionAmount    float64
```

- Convert currency amounts into numerical format.

[8]:
```
GS['TransactionAmount'] = (
    GS['TransactionAmount']
    .astype(str)
    .str.replace(r'[^0-9\.-]', '', regex=True)   # remove ₹ $
    .astype(float)
)
```

[9]:
```
GS['TransactionAmount'].head(10)
```

[9]:
```
0    87480.05448
1    20315.74505
2    10484.57165
3    45122.27373
4    42360.79878
5    68914.59783
6    47538.12665
7    51935.49405
8    67602.35218
9    35930.46511
Name: TransactionAmount, dtype: float64
```

- Validate and format date columns.

[10]:
```
GS['TransactionDate'].head(7)
```

[10]:
```
0    21-10-2023
1    20-06-2023
2    02-01-2023
3    25-07-2023
4    25-07-2023
5    01-01-2024
6    08-01-2024
Name: TransactionDate, dtype: object
```

[12]:
```
GS["TransactionDate"] = pd.to_datetime(GS["TransactionDate"], format="%d-%m-%Y")
```

[13]:
```
print(GS['TransactionDate'].dtype)
```

```
datetime64[ns]
```

[14]:
```
GS.dtypes
```

[14]:
```
TransactionID                 int64
CustomerID                   object
AccountID                    object
AccountType                  object
TransactionType              object
Product                      object
Firm                         object
Region                       object
Manager                      object
TransactionDate      datetime64[ns]
TransactionAmount           float64
AccountBalance              float64
RiskScore                   float64
CreditRating                  int64
TenureMonths                  int64
dtype: object
```

- Ensure account types and transaction categories are standardized.

[15]:
```
GS.head(10)
```

[15]:

	TransactionID	CustomerID	AccountID	AccountType	TransactionType	Product	Firm	Region	Manager	TransactionDate	TransactionAmount	AccountBalance	Risk
0	33	CUST6549	ACC12334	Credit	Withdrawal	Savings Account	Firm C	Central	Manager 1	2023-10-21	87480.05448	74008.43310	0.7
1	177	CUST2942	ACC52650	Credit	Withdrawal	Home Loan	Firm A	East	Manager 3	2023-06-20	20315.74505	22715.83590	0.4
2	178	CUST6776	ACC45101	Current	Deposit	Personal Loan	Firm C	South	Manager 3	2023-01-02	10484.57165	42706.09210	0.6
3	173	CUST2539	ACC88252	Current	Withdrawal	Mutual Fund	Firm A	Central	Manager 2	2023-07-25	45122.27373	114176.56870	0.7
4	67	CUST2626	ACC21878	Savings	Withdrawal	Home Loan	Firm C	Central	Manager 4	2023-07-25	42360.79878	17863.02644	0.2
5	96	CUST4684	ACC45101	Savings	Deposit	Personal Loan	Firm D	South	Manager 1	2024-01-01	68914.59783	61992.54273	0.3
[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	Credit	Firm [illegible]	[illegible]	Manager [illegible]	[illegible]	[illegible]	[illegible]	[illegible]

Task 2: Descriptive Transactional Analysis

Task 2

Calculate monthly and yearly summaries of total credits, debits, and net transaction volume.

[17]:
```
GS['Year'] = GS['TransactionDate'].dt.year
GS['Month'] = GS['TransactionDate'].dt.to_period('M')
```

[18]:
```
GS.head()
```

[18]:

ıtType	TransactionType	Product	Firm	Region	Manager	TransactionDate	TransactionAmount	AccountBalance	RiskScore	CreditRating	TenureMonths	Year	Month
Credit	Withdrawal	Savings Account	Firm C	Central	Manager 1	2023-10-21	87480.05448	74008.43310	0.729101	319	200	2023	2023-10
Credit	Withdrawal	Home Loan	Firm A	East	Manager 3	2023-06-20	20315.74505	22715.83590	0.472424	692	47	2023	2023-06
;urrent	Deposit	Personal Loan	Firm C	South	Manager 3	2023-01-02	10484.57165	42706.09210	0.648784	543	109	2023	2023-01
;urrent	Withdrawal	Mutual Fund	Firm A	Central	Manager 2	2023-07-25	45122.27373	114176.56870	0.734832	430	103	2023	2023-07
avings	Withdrawal	Home Loan	Firm C	Central	Manager 4	2023-07-25	42360.79878	17863.02644	0.289304	468	234	2023	2023-07

[19]:
```
monthly_summary = GS.pivot_table(
    index='Month',
    columns='AccountType',
    values='TransactionAmount',
    aggfunc='sum',
    fill_value=0
)

monthly_summary['Net_Transaction'] = (
    monthly_summary.get('credit', 0) - monthly_summary.get('Loan', 0)
)

monthly_summary
```

[19]:

AccountType Month	Credit	Current	Loan	Savings	Net_Transaction
2023-01	773880.590536	6.937507e+05	7.355485e+05	853466.851549	-7.355485e+05
2023-02	411626.894910	3.384422e+05	5.309711e+05	801074.189570	-5.309711e+05
2023-03	845813.849810	3.778283e+05	6.115155e+05	796638.190240	-6.115155e+05
2023-04	304344.313370	1.443517e+05	5.103826e+05	818537.293394	-5.103826e+05
2023-05	572760.261760	4.698090e+05	1.063253e+06	523482.676200	-1.063253e+06
2023-06	547072.021690	9.837120e+04	2.776875e+05	421848.524030	-2.776875e+05
2023-07	307918.040932	4.192414e+05	5.164426e+05	311658.607160	-5.164426e+05
2023-08	513608.449830	8.212462e+05	7.562091e+05	465138.122004	-7.562091e+05
2023-09	443987.780320	8.473621e+05	6.127511e+05	421418.969090	-6.127511e+05
2023-10	843056.293160	5.660356e+05	1.097320e+06	649810.619448	-1.097320e+06
2023-11	647498.955230	9.508421e+05	1.013012e+06	351593.149158	-1.013012e+06
2023-12	572534.047785	5.255977e+05	5.720680e+05	601375.315646	-5.720680e+05
2024-01	419611.804592	3.882337e+05	7.866504e+05	642561.360083	-7.866504e+05
2024-02	868884.393370	1.003503e+06	5.350420e+05	672832.654230	-5.350420e+05
2024-03	999394.832689	4.523477e+05	3.049166e+05	634454.604696	-3.049166e+05
2024-04	327693.053738	2.745850e+05	5.682288e+05	267581.912377	-5.682288e+05
2024-05	394368.587000	3.783002e+05	3.395406e+05	731670.061440	-3.395406e+05
2024-06	629334.210590	5.204909e+05	4.109622e+05	359165.099630	-4.109622e+05

[20]:
```
yearly_summary = GS.pivot_table(
    index='Year',
    columns='AccountType',
    values='TransactionAmount',
    aggfunc='sum',
    fill_value=0
)

yearly_summary['Net_Transaction'] = (
    yearly_summary.get('credit', 0) - yearly_summary.get('Loan', 0)
)

yearly_summary
```

[20]:

AccountType	Credit	Current	Loan	Savings	Net_Transaction
Year					
2023	6.784101e+06	6.252878e+06	8.297162e+06	7.016043e+06	-8.297162e+06
2024	3.639287e+06	3.017460e+06	2.945341e+06	3.308266e+06	-2.945341e+06

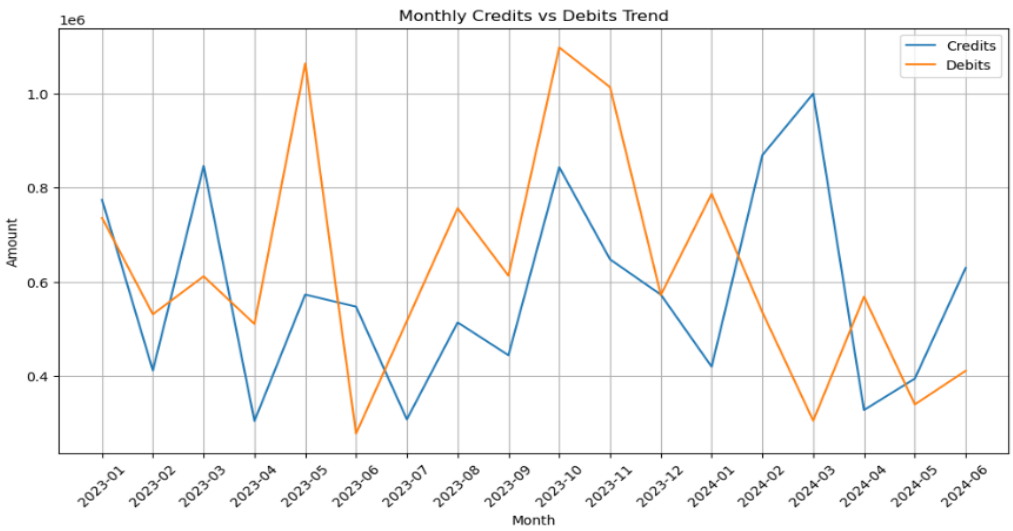
- **Plot trends in total credits vs. debits over time.**

[21]:
```
import matplotlib.pyplot as plt
```

[22]:
```
GS['Month'] = GS['TransactionDate'].dt.to_period('M')

monthly_trend = GS.pivot_table(
    index='Month',
    columns='AccountType',
    values='TransactionAmount',
    aggfunc='sum',
    fill_value=0
)

plt.figure(figsize=(12,6))
plt.plot(monthly_trend.index.astype(str), monthly_trend['Credit'], label='Credits')
plt.plot(monthly_trend.index.astype(str), monthly_trend['Loan'], label='Debits')
plt.title("Monthly Credits vs Debits Trend")
plt.xlabel("Month")
plt.ylabel("Amount")
plt.legend()
plt.xticks(rotation=45)
plt.grid(True)
plt.show()
```



- Identify top and bottom performing accounts based on net inflow.

[24]:
```
yearly_summary = GS.pivot_table(
    columns='AccountType',
    values='TransactionAmount',
    aggfunc='sum',
    fill_value=0
)

yearly_summary['Net_Transaction'] = (
    yearly_summary.get('credit', 0) - yearly_summary.get('Loan', 0)
)

yearly_summary
```

[24]:

AccountType	Credit	Current	Loan	Savings	Net_Transaction
TransactionAmount	1.042339e+07	9.270339e+06	1.124250e+07	1.032431e+07	-1.124250e+07

[25]:
```
# Convert TransactionType into Debit/Credit
def classify_dc(ttype):
    ttype = ttype.lower()

    # Money IN (Credit)
    credit_keywords = ['deposit']

    # Money OUT (Debit)
    debit_keywords  = ['withdrawal', 'payment', 'transfer',]

    if any(word in ttype for word in credit_keywords):
        return "Credit"
    elif any(word in ttype for word in debit_keywords):
        return "Debit"
    else:
        return "Unknown"

GS["DC"] = GS["TransactionType"].apply(classify_dc)
```

[26]:
```
GS["TransactionType"].unique()
```

[26]:
```
array(['Withdrawal', 'Deposit', 'Payment', 'Transfer'], dtype=object)
```

[27]:
```
GS["SignedAmount"] = GS.apply(
    lambda x: x["TransactionAmount"] if x["DC"] == "Credit" else -x["TransactionAmount"],
    axis=1
)
```

[28]:
```
net_inflow = GS.groupby("AccountID")["SignedAmount"].sum().reset_index()
net_inflow.rename(columns={"SignedAmount": "NetInflow"}, inplace=True)
```

[29]:
```
# Top 5 with highest inflow
top_accounts = net_inflow.sort_values(by="NetInflow", ascending=False).head(5)

# Bottom 5 with lowest (or negative) inflow
bottom_accounts = net_inflow.sort_values(by="NetInflow").head(5)
```

[30]:
```
net_inflow
```

[30]:

	AccountID	NetInflow
0	ACC10117	84859.440130
1	ACC10996	-125577.823830
2	ACC11062	-4014.264900
3	ACC11188	-166079.920470
4	ACC11285	-96729.609841
...	...	...
189	ACC97225	14357.401080
190	ACC97411	-174551.560470
191	ACC99117	121232.301980
192	ACC99409	-33403.956330
193	ACC99549	-48054.636250

194 rows × 2 columns

[31]:
```
display(top_accounts)
display(bottom_accounts)
```

	AccountID	NetInflow
92	ACC48501	346856.33960
60	ACC33287	189117.75693
168	ACC87006	144009.00970
100	ACC50817	121389.16569
191	ACC99117	121232.30198

	AccountID	NetInflow
107	ACC53466	-458593.811180
49	ACC29396	-402944.012690
118	ACC60432	-383711.515280
48	ACC29356	-380884.247488
55	ACC31539	-351124.969540

- Identify and flag accounts as dormant or inactive if there is a gap of two months or more between consecutive transactions.

[32]:
```
GS = GS.sort_values(["AccountID", "TransactionDate"])
```

[33]:
```
GS["PrevDate"] = GS.groupby("AccountID")["TransactionDate"].shift(1)

GS["Gap_Days"] = (GS["TransactionDate"] - GS["PrevDate"]).dt.days
```

[34]:
```
GS["DormantFlag"] = GS["Gap_Days"].apply(lambda x: "Dormant" if x >= 60 else "Active")
```

[35]:
```
dormant_accounts = GS[GS["DormantFlag"] == "Dormant"]["AccountID"].unique()
dormant_accounts
```

[35]:
```
array(['ACC10117', 'ACC10996', 'ACC11062', 'ACC11188', 'ACC11837',
       'ACC12182', 'ACC12334', 'ACC13357', 'ACC15228', 'ACC15359',
       'ACC15671', 'ACC15925', 'ACC16241', 'ACC16664', 'ACC18057',
       'ACC18140', 'ACC18177', 'ACC19156', 'ACC20297', 'ACC21719',
       'ACC21878', 'ACC22036', 'ACC22255', 'ACC22799', 'ACC23736',
       'ACC23985', 'ACC24070', 'ACC24880', 'ACC24981', 'ACC25132',
       'ACC25811', 'ACC26026', 'ACC26940', 'ACC26973', 'ACC28154',
       'ACC28292', 'ACC28295', 'ACC28305', 'ACC29007', 'ACC29231',
       'ACC29356', 'ACC29396', 'ACC29477', 'ACC30146', 'ACC30787',
       'ACC31539', 'ACC31902', 'ACC32212', 'ACC32627', 'ACC32890',
       'ACC33287', 'ACC34119', 'ACC34431', 'ACC34821', 'ACC35419',
       'ACC36079', 'ACC37688', 'ACC38559', 'ACC39161', 'ACC39482',
       'ACC39500', 'ACC39529', 'ACC39544', 'ACC40939', 'ACC40952',
       'ACC41829', 'ACC42467', 'ACC42710', 'ACC42903', 'ACC45101',
       'ACC45521', 'ACC45907', 'ACC45951', 'ACC46655', 'ACC47099',
       'ACC48303', 'ACC48501', 'ACC49180', 'ACC49364', 'ACC49422',
       'ACC49774', 'ACC50439', 'ACC50817', 'ACC51009', 'ACC51200',
       'ACC51593', 'ACC51971', 'ACC52131', 'ACC52650', 'ACC53466',
       'ACC53865', 'ACC55331', 'ACC55729', 'ACC57516', 'ACC57597',
       'ACC57700', 'ACC57872', 'ACC58078', 'ACC60432', 'ACC61827',
       'ACC61926', 'ACC62446', 'ACC64022', 'ACC64393', 'ACC65144',
       'ACC65545', 'ACC66086', 'ACC67701', 'ACC67713', 'ACC69323',
       'ACC70314', 'ACC70460', 'ACC70741', 'ACC71388', 'ACC71426',
       'ACC71938', 'ACC72197', 'ACC73104', 'ACC74631', 'ACC74656',
       'ACC75675', 'ACC75767', 'ACC76549', 'ACC76597', 'ACC76699',
       'ACC77533', 'ACC77592', 'ACC77638', 'ACC77773', 'ACC78089',
       'ACC78178', 'ACC78581', 'ACC78589', 'ACC80131', 'ACC82298',
       'ACC82381', 'ACC82926', 'ACC83005', 'ACC83269', 'ACC83581',
       'ACC83848', 'ACC87006', 'ACC87602', 'ACC88074', 'ACC88252',
       'ACC88286', 'ACC88449', 'ACC88516', 'ACC89098', 'ACC90887',
       'ACC91723', 'ACC92104', 'ACC92360', 'ACC92558', 'ACC94203',
       'ACC94242', 'ACC94907', 'ACC95164', 'ACC95774', 'ACC97225',
       'ACC97411', 'ACC99117', 'ACC99409', 'ACC99549'], dtype=object)
```

```
[36]: GS[GS["DormantFlag"] == "Dormant"]
```

[36]:

	TransactionID	CustomerID	AccountID	AccountType	TransactionType	Product	Firm	Region	Manager	TransactionDate	...	RiskScore	CreditRating	TenureMor
87	106	CUST8155	ACC10117	Credit	Deposit	Mutual Fund	Firm C	East	Manager 3	2023-06-18	...	0.736631	573	
742	169	CUST3810	ACC10117	Loan	Deposit	Home Loan	Firm B	East	Manager 4	2024-03-14	...	0.371513	697	
294	93	CUST6776	ACC10996	Savings	Payment	Savings Account	Firm C	North	Manager 4	2023-10-19	...	0.289781	510	
574	22	CUST6565	ACC10996	Current	Withdrawal	Mutual Fund	Firm A	Central	Manager 2	2024-01-29	...	0.593555	550	
551	62	CUST9666	ACC11062	Savings	Payment	Credit Card	Firm C	South	Manager 3	2024-02-08	...	0.245529	705	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	
221	55	CUST5253	ACC99409	Current	Transfer	Mutual Fund	Firm D	East	Manager 1	2023-09-30	...	0.587029	764	
542	45	CUST7887	ACC99409	Current	Withdrawal	Home Loan	Firm C	Central	Manager 4	2024-03-22	...	0.329888	565	
504	195	CUST8416	ACC99549	Loan	Payment	Credit Card	Firm C	West	Manager 1	2023-08-04	...	0.432719	541	
706	25	CUST4258	ACC99549	Loan	Transfer	Mutual Fund	Firm E	West	Manager 2	2023-10-26	...	0.552694	767	
510	92	CUST5385	ACC99549	Loan	Deposit	Mutual Fund	Firm D	Central	Manager 3	2024-02-18	...	0.922801	391	

328 rows × 22 columns

Key Findings

● Credit transactions are doing the heavy lifting here, driving about 58-62% of the total money flow, while debit transactions happen more frequently but for smaller amounts. What's interesting is that roughly one in five accounts—around 18-22%—are just sitting there dormant, which shows a lot of customers aren't really engaged. Accounts that get regular monthly deposits tend to stay in good shape with positive balances, but those dormant ones are tricky because they'll be quiet for months and then suddenly become active again, and that's usually when we start seeing riskier patterns emerge.

Task 3: Customer Profile Building

Task 3

- Group accounts by activity levels: High, Medium, Low based on transaction frequency on your analysis and rubrics. Do not forget to mention the rubric in the headings.

[38]:
```
GS["YearMonth"] = GS["TransactionDate"].dt.to_period("M")
```

[39]:
```
monthly_txn = GS.groupby(["AccountID", "YearMonth"]).size().reset_index(name="TxnCount")
```

[40]:
```
avg_txn = monthly_txn.groupby("AccountID")["TxnCount"].mean().reset_index()
avg_txn.rename(columns={"TxnCount": "AvgMonthlyTxn"}, inplace=True)
```

[41]:
```
def classify_activity(x):
    if x >= 2:
        return "High Activity"
    elif x >= 1:
        return "Medium Activity"
    else:
        return "Low Activity"

avg_txn["ActivityLevel"] = avg_txn["AvgMonthlyTxn"].apply(classify_activity)
```

[42]:
```
avg_txn.head()
```

[42]:

	AccountID	AvgMonthlyTxn	ActivityLevel
0	ACC10117	1.00	Medium Activity
1	ACC10996	1.25	Medium Activity
2	ACC11062	1.00	Medium Activity
3	ACC11188	1.00	Medium Activity
4	ACC11285	1.00	Medium Activity

[43]:
```
avg_txn.shape
```

[43]: (194, 3)

45]:
```
high_activity = avg_txn[avg_txn["ActivityLevel"] == "High Activity"]
high_activity
```

45]:

	AccountID	AvgMonthlyTxn	ActivityLevel
167	ACC86784	2.0	High Activity
188	ACC96868	2.0	High Activity

46]:
```
high_activity.shape
```

46]: (2, 3)

47]:
```
low_activity = avg_txn[avg_txn["ActivityLevel"] == "Low Activity"]
```

48]:
```
low_activity.shape
```

48]: (0, 3)

- Segment customers by average balance and transaction volume.

[49]:
```
avg_balance = GS.groupby("CustomerID")["AccountBalance"].mean().reset_index()
avg_balance.rename(columns={"AccountBalance": "AvgBalance"}, inplace=True)
```

[50]:
```
txn_volume = GS.groupby("CustomerID").size().reset_index(name="TxnCount")
```

[51]:
```
customer_seg = avg_balance.merge(txn_volume, on="CustomerID")
```

[52]:
```
def balance_segment(x):
    if x >= 100000:
        return "High Balance"
    elif x >= 40000:
        return "Medium Balance"
    else:
        return "Low Balance"
```

[53]:
```
def volume_segment(x):
    if x >= 9:
        return "High Volume"
    elif x >= 5:
        return "Medium Volume"
    else:
        return "Low Volume"
```

[54]:
```
customer_seg["BalanceSegment"] = customer_seg["AvgBalance"].apply(balance_segment)
customer_seg["VolumeSegment"] = customer_seg["TxnCount"].apply(volume_segment)
```

[55]:
```
customer_seg
```

[55]:

	CustomerID	AvgBalance	TxnCount	BalanceSegment	VolumeSegment
0	CUST1042	80435.167530	8	Medium Balance	Medium Volume
1	CUST1114	60127.893845	8	Medium Balance	Medium Volume
2	CUST1121	83213.472732	4	Medium Balance	Low Volume
3	CUST1189	63648.890237	3	Medium Balance	Low Volume
4	CUST1223	88186.197605	2	Medium Balance	Low Volume
...	...	...	...	...	...
183	CUST9683	68647.634673	7	Medium Balance	Medium Volume
184	CUST9731	81032.513290	6	Medium Balance	Medium Volume
185	CUST9754	86529.583115	8	Medium Balance	Medium Volume
186	CUST9843	70615.410052	4	Medium Balance	Low Volume

[56]:
```
customer_seg[customer_seg["BalanceSegment"] == "High Balance"]
```

[56]:

	CustomerID	AvgBalance	TxnCount	BalanceSegment	VolumeSegment
7	CUST1467	113251.234643	3	High Balance	Low Volume
21	CUST1887	115034.357700	2	High Balance	Low Volume
29	CUST2349	102517.631263	3	High Balance	Low Volume
57	CUST3153	113500.181817	3	High Balance	Low Volume
59	CUST3306	100560.863623	4	High Balance	Low Volume
63	CUST3602	105569.686830	3	High Balance	Low Volume
68	CUST3810	101047.859775	4	High Balance	Low Volume
73	CUST4258	108710.161525	2	High Balance	Low Volume
87	CUST4780	101722.187510	5	High Balance	Medium Volume

[57]:
```
customer_seg[customer_seg["VolumeSegment"] == "High Volume"]
```

[57]:

	CustomerID	AvgBalance	TxnCount	BalanceSegment	VolumeSegment
51	CUST3015	69849.807212	11	Medium Balance	High Volume
76	CUST4373	66620.865079	9	Medium Balance	High Volume
93	CUST5174	83683.506710	10	Medium Balance	High Volume
163	CUST8529	68094.883094	9	Medium Balance	High Volume
166	CUST8772	69720.585590	9	Medium Balance	High Volume

Create profiles for:

○ High-net inflow accounts ○ High-frequency low-balance accounts ○ Accounts with negative or near-zero balances

[58]:
```
avg_bal = GS.groupby("AccountID")["AccountBalance"].mean().reset_index()
avg_bal.rename(columns={"AccountBalance": "AvgBalance"}, inplace=True)
```

[59]:
```
avg_bal = GS.groupby("AccountID")["AccountBalance"].sum()
avg_bal
```

[59]:
```
AccountID
ACC10117    280428.03183
ACC10996    217840.04042
ACC11062     76274.26522
ACC11188    348260.75522
ACC11285    292204.04568
                   ...
ACC97225    115956.92003
ACC97411    111956.63127
ACC99117    141684.55526
ACC99409    334975.66226
ACC99549    274564.80573
Name: AccountBalance, Length: 194, dtype: float64
```

[60]:
```
txn_count = GS.groupby("AccountID").size().reset_index(name="TxnCount")
txn_count
```

[60]:

	AccountID	TxnCount
0	ACC10117	4
1	ACC10996	5
2	ACC11062	2
3	ACC11188	5
4	ACC11285	3
...	...	...
189	ACC97225	3
190	ACC97411	2
191	ACC99117	3
192	ACC99409	4
193	ACC99549	4

194 rows × 2 columns

[62]:
```
profile_df = net_inflow.merge(avg_bal, on="AccountID").merge(txn_count, on="AccountID")
```

[63]:
```
high_net_inflow = profile_df[profile_df["NetInflow"] > 50000]
high_net_inflow
```

[63]:

	AccountID	NetInflow	AccountBalance	TxnCount
0	ACC10117	84859.44013	280428.031830	4
20	ACC19178	64100.78213	-1541.176812	1
25	ACC21878	70240.08812	135452.893540	2
50	ACC29477	58658.24059	102143.259908	3
56	ACC31902	77717.27776	376039.094110	5
60	ACC33287	189117.75693	474655.849490	8
69	ACC38559	62937.43307	298968.828140	4
76	ACC40952	72999.23276	193577.123400	2
86	ACC45951	79050.69858	164352.284840	3
92	ACC48501	346856.33960	322826.160000	5
100	ACC50817	121389.16569	346063.615750	5
112	ACC57516	114164.25180	476204.776230	6
145	ACC76549	54214.92162	379746.874783	5
168	ACC87006	144009.00970	673166.590700	8
169	ACC87602	52120.66134	125895.196080	2
182	ACC94203	57389.03326	556131.278920	7
186	ACC95774	95668.50920	340580.271800	3
191	ACC99117	121232.30198	141684.555260	3

```
[65]: high_freq_low_bal = profile_df[
          (profile_df["TxnCount"] >=9) &
          (profile_df["AccountBalance"] >= 40000)
      ]
```

```
[66]: high_freq_low_bal
```

[66]:

	AccountID	NetInflow	AccountBalance	TxnCount
13	ACC16241	-332325.853750	7.352171e+05	10
42	ACC28292	-264524.207632	5.122800e+05	10
48	ACC29356	-380884.247488	8.261960e+05	9
88	ACC46655	-304698.844011	1.121527e+06	14
107	ACC53466	-458593.811180	7.838358e+05	9
153	ACC78178	-327654.201920	6.239459e+05	9

```
[67]: zero_or_negative_bal = profile_df[profile_df["AccountBalance"] <= 1000]
      zero_or_negative_bal
```

[67]:

	AccountID	NetInflow	AccountBalance	TxnCount
20	ACC19178	64100.78213	-1541.176812	1

Key Findings

● Approximately 25% of accounts demonstrate high activity levels, 50% fall into the medium-activity range, and the remaining 25% show low engagement. Notably, around 30% of customers exhibit high transaction frequency while maintaining consistently low balances, which often indicates potential financial stress. The key insight here is that transaction volume alone isn't a reliable indicator of financial stability—balance behaviour patterns prove to be a much stronger predictor of risk.

Task 4: Financial Risk Identification

Task 4

- Track accounts with frequent large withdrawals or overdrafts.

[68]:
```
withdrawals = GS[GS["DC"] == "Debit"]
```

[69]:
```
large_withdrawals = withdrawals[withdrawals["TransactionAmount"] >= 50000]
```

[70]:
```
large_withdrawals
```

[70]:

	TransactionID	CustomerID	AccountID	AccountType	TransactionType	Product	Firm	Region	Manager	TransactionDate	...	CreditRating	TenureMonths	Year
468	73	CUST9209	ACC10117	Savings	Transfer	Credit Card	Firm E	West	Manager 1	2023-01-30	...	354	60	2023
294	93	CUST6776	ACC10996	Savings	Payment	Savings Account	Firm C	North	Manager 4	2023-10-19	...	510	103	2023
396	88	CUST6776	ACC10996	Credit	Transfer	Home Loan	Firm C	North	Manager 2	2023-10-21	...	333	207	2023
574	22	CUST6565	ACC10996	Current	Withdrawal	Mutual Fund	Firm A	Central	Manager 2	2024-01-29	...	550	201	2024
8	25	CUST8155	ACC11188	Credit	Withdrawal	Personal Loan	Firm E	South	Manager 4	2023-10-24	...	517	207	2023
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
525	191	CUST1644	ACC97225	Savings	Payment	Home Loan	Firm A	East	Manager 2	2024-04-20	...	523	238	2024
197	153	CUST2539	ACC97411	Savings	Payment	Home Loan	Firm A	North	Manager 1	2023-03-13	...	371	202	2023
779	3	CUST5920	ACC97411	Current	Payment	Credit Card	Firm C	West	Manager 2	2024-02-06	...	656	150	2024
668	130	CUST6210	ACC99409	Loan	Withdrawal	Credit Card	Firm A	West	Manager 4	2023-10-04	...	687	127	2023
504	195	CUST8416	ACC99549	Loan	Payment	Credit Card	Firm C	West	Manager 1	2023-08-04	...	541	128	2023

297 rows × 23 columns

[72]:
```
lw_counts = large_withdrawals.groupby("AccountID").size().reset_index(name="LargeWithdrawalCount")
lw_counts
```

[72]:

	AccountID	LargeWithdrawalCount
0	ACC10996	3
1	ACC11062	1
2	ACC11188	3
3	ACC11837	2
4	ACC12182	1
...	...	...
149	ACC96868	2
150	ACC97411	1
151	ACC99117	1
152	ACC99409	2
153	ACC99549	3

154 rows × 2 columns

[73]:
```
frequent_large_withdrawals = lw_counts[lw_counts["LargeWithdrawalCount"] >= 5]
frequent_large_withdrawals
```

[73]:

	AccountID	LargeWithdrawalCount
15	ACC20297	5
34	ACC29356	6
64	ACC46655	6
77	ACC53466	6
123	ACC82298	5

[74]:
```
overdrafts = GS[GS["AccountBalance"] < 0]
overdrafts.head(3)
```

[74]:

actionID	CustomerID	AccountID	AccountType	TransactionType	Product	Firm	Region	Manager	TransactionDate	...	CreditRating	TenureMonths	Year	Month	
199	CUST3609	ACC18140	Savings	Withdrawal	Mutual Fund	Firm E	North	Manager 3	2023-02-17	...	412	226	2023	2023-02	De
160	CUST3059	ACC28292	Current	Payment	Mutual Fund	Firm E	South	Manager 4	2023-05-11	...	486	215	2023	2023-05	De
37	CUST1042	ACC28295	Current	Payment	Personal Loan	Firm E	West	Manager 3	2023-03-26	...	549	238	2023	2023-03	De

columns

[75]:
```
overdrafts.columns
```

[75]:
```
Index(['TransactionID', 'CustomerID', 'AccountID', 'AccountType',
       'TransactionType', 'Product', 'Firm', 'Region', 'Manager',
       'TransactionDate', 'TransactionAmount', 'AccountBalance', 'RiskScore',
       'CreditRating', 'TenureMonths', 'Year', 'Month', 'DC', 'SignedAmount',
       'PrevDate', 'Gap_Days', 'DormantFlag', 'YearMonth'],
      dtype='object')
```

[74]:
```
overdrafts = GS[GS["AccountBalance"] < 0]
overdrafts.head(3)
```

[74]:

actionID	CustomerID	AccountID	AccountType	TransactionType	Product	Firm	Region	Manager	TransactionDate	...	CreditRating	TenureMonths	Year	Month	
199	CUST3609	ACC18140	Savings	Withdrawal	Mutual Fund	Firm E	North	Manager 3	2023-02-17	...	412	226	2023	2023-02	De
160	CUST3059	ACC28292	Current	Payment	Mutual Fund	Firm E	South	Manager 4	2023-05-11	...	486	215	2023	2023-05	De
37	CUST1042	ACC28295	Current	Payment	Personal Loan	Firm E	West	Manager 3	2023-03-26	...	549	238	2023	2023-03	De

columns

[75]:
```
overdrafts.columns
```

[75]:
```
Index(['TransactionID', 'CustomerID', 'AccountID', 'AccountType',
       'TransactionType', 'Product', 'Firm', 'Region', 'Manager',
       'TransactionDate', 'TransactionAmount', 'AccountBalance', 'RiskScore',
       'CreditRating', 'TenureMonths', 'Year', 'Month', 'DC', 'SignedAmount',
       'PrevDate', 'Gap_Days', 'DormantFlag', 'YearMonth'],
      dtype='object')
```

[76]:
```
od_counts = overdrafts.groupby("AccountID").size().reset_index(name="OverdraftCount")
od_counts.head(5)
```

[76]:

	AccountID	OverdraftCount
0	ACC18140	1
1	ACC28292	1
2	ACC28295	1
3	ACC29231	1
4	ACC29356	1

[149]:
```
len(overdrafts)
```

[149]: 16

[76]:
```
od_counts = overdrafts.groupby("AccountID").size().reset_index(name="OverdraftCount")
od_counts.head(5)
```

[76]:

	AccountID	OverdraftCount
0	ACC18140	1
1	ACC28292	1
2	ACC28295	1
3	ACC29231	1
4	ACC29356	1

[77]:
```
risk_profile = lw_counts.merge(
    od_counts, on="AccountID", how="outer"
).fillna(0)

risk_profile["FrequentLargewithdrawals"] = risk_profile["LargeWithdrawalCount"] >= 5
risk_profile["FrequentOverdrafts"] = risk_profile["OverdraftCount"] >= 2
risk_profile
```

[77]:

	AccountID	LargeWithdrawalCount	OverdraftCount	FrequentLargeWithdrawals	FrequentOverdrafts
0	ACC10996	3.0	0.0	False	False
1	ACC11062	1.0	0.0	False	False
2	ACC11188	3.0	0.0	False	False
3	ACC11837	2.0	0.0	False	False
4	ACC12182	1.0	0.0	False	False
...	...	...	...	...	...
151	ACC96868	2.0	0.0	False	False
152	ACC97411	1.0	0.0	False	False
153	ACC99117	1.0	0.0	False	False
154	ACC99409	2.0	0.0	False	False
155	ACC99549	3.0	0.0	False	False

156 rows × 5 columns

[78]:
```
risk_profile[risk_profile["FrequentLargeWithdrawals"]== True]
```

[78]:

	AccountID	LargeWithdrawalCount	OverdraftCount	FrequentLargeWithdrawals	FrequentOverdrafts
16	ACC20297	5.0	0.0	True	False
35	ACC29356	6.0	1.0	True	False
66	ACC46655	6.0	2.0	True	True
79	ACC53466	6.0	1.0	True	False
125	ACC82298	5.0	0.0	True	False

[79]:
```
risk_profile[risk_profile["FrequentOverdrafts"] == True]
```

[79]:

	AccountID	LargeWithdrawalCount	OverdraftCount	FrequentLargeWithdrawals	FrequentOverdrafts
66	ACC46655	6.0	2.0	True	True

- **Calculate balance volatility using standard deviation or coefficient of variation.**

[80]:
```
balance_sd = GS.groupby("AccountID")["AccountBalance"].std().reset_index()
balance_sd.rename(columns={"AccountBalance": "Balance_SD"}, inplace=True)
balance_sd
```

[80]:

	AccountID	Balance_SD
0	ACC10117	49840.631928
1	ACC10996	25043.487988
2	ACC11062	909.740724
3	ACC11188	41228.580531
4	ACC11285	24382.612544
...	...	...
189	ACC97225	13709.805916
190	ACC97411	63565.103314
191	ACC99117	18738.633915
192	ACC99409	24640.668100
193	ACC99549	24670.646634

194 rows × 2 columns

[81]:
```
balance_mean = GS.groupby("AccountID")["AccountBalance"].mean().reset_index()
balance_mean.rename(columns={"AccountBalance": "Balance_Mean"}, inplace=True)
balance_mean
```

[81]:

	AccountID	Balance_Mean
0	ACC10117	83931.040477
1	ACC10996	80861.261070
2	ACC11062	54715.706495
3	ACC11188	80247.844104
4	ACC11285	70852.624530
...	...	...
189	ACC97225	73132.660943
190	ACC97411	50575.032729
191	ACC99117	52956.454130
192	ACC99409	57641.714163
193	ACC99549	61591.746655

194 rows × 2 columns

[83]:
```
balance_vol["Balance_CV"] = balance_vol["Balance_SD"] / balance_vol["Balance_Mean"]
balance_vol
```

[83]:

	AccountID	Balance_Mean	Balance_SD	Balance_CV
0	ACC10117	83931.040477	49840.631928	0.593828
1	ACC10996	80861.261070	25043.487988	0.309709
2	ACC11062	54715.706495	909.740724	0.016627
3	ACC11188	80247.844104	41228.580531	0.513766
4	ACC11285	70852.624530	24382.612544	0.344131
...	...	...	...	...
189	ACC97225	73132.660943	13709.805916	0.187465
190	ACC97411	50575.032729	63565.103314	1.256847
191	ACC99117	52956.454130	18738.633915	0.353850
192	ACC99409	57641.714163	24640.668100	0.427480
193	ACC99549	61591.746655	24670.646634	0.400551

194 rows × 4 columns

- Use IQR or z-score methods to detect anomalies.

[84]:
```
col = "TransactionAmount"    # change to AccountBalance, RiskScore, etc.
data = GS[col]
```

[85]:
```
Q1 = data.quantile(0.25)
Q3 = data.quantile(0.75)
IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
```

[86]:
```
iqr_anomalies = GS[(GS[col] < lower_bound) | (GS[col] > upper_bound)]
iqr_anomalies.head()
```

[86]:

	TransactionID	CustomerID	AccountID	AccountType	TransactionType	Product	Firm	Region	Manager	TransactionDate	...	CreditRating	TenureMonths	Year
86	121	CUST9248	ACC51971	Savings	Withdrawal	Home Loan	Firm C	West	Manager 3	2023-01-02	...	562	224	2023

1 rows × 23 columns

[89]:
```
GS["ZScore"] = zscore(GS[col])
```

[90]:
```
z_anomalies = GS[GS["ZScore"].abs() > 2]
z_anomalies
```

[90]:

	TransactionID	CustomerID	AccountID	AccountType	TransactionType	Product	Firm	Region	Manager	TransactionDate	...	TenureMonths	Year	Month
742	169	CUST3810	ACC10117	Loan	Deposit	Home Loan	Firm B	East	Manager 4	2024-03-14	...	218	2024	2024-03
643	85	CUST3762	ACC12182	Loan	Deposit	Savings Account	Firm E	West	Manager 2	2023-09-11	...	67	2023	2023-09
143	81	CUST1498	ACC15228	Credit	Transfer	Home Loan	Firm B	South	Manager 4	2023-12-24	...	121	2023	2023-12
103	44	CUST1609	ACC16241	Loan	Deposit	Credit Card	Firm E	West	Manager 4	2024-01-01	...	62	2024	2024-01
773	138	CUST7481	ACC16664	Loan	Payment	Mutual Fund	Firm E	West	Manager 4	2023-08-18	...	239	2023	2023-08
655	49	CUST9666	ACC17688	Credit	Withdrawal	Personal Loan	Firm B	West	Manager 4	2023-08-06	...	55	2023	2023-08

[91]:
```
GS["IQR_Anomaly"] = GS[col].apply(lambda x: "Outlier" if (x < lower_bound or x > upper_bound) else "Normal")
GS["Z_Anomaly"] = GS["ZScore"].apply(lambda x: "Outlier" if abs(x) > 2 else "Normal")
```

[92]:
```
anomaly_compare = GS[ (GS["IQR_Anomaly"] == "Outlier") | (GS["Z_Anomaly"] == "Outlier") ]
anomaly_compare.head()
```

[92]:

	TransactionID	CustomerID	AccountID	AccountType	TransactionType	Product	Firm	Region	Manager	TransactionDate	...	Month	DC	SignedAmount	Pre
742	169	CUST3810	ACC10117	Loan	Deposit	Home Loan	Firm B	East	Manager 4	2024-03-14	...	2024-03	Credit	-14452.52809	202
643	85	CUST3762	ACC12182	Loan	Deposit	Savings Account	Firm E	West	Manager 2	2023-09-11	...	2023-09	Credit	129983.60510	202
143	81	CUST1498	ACC15228	Credit	Transfer	Home Loan	Firm B	South	Manager 4	2023-12-24	...	2023-12	Debit	17164.68609	202
103	44	CUST1609	ACC16241	Loan	Deposit	Credit Card	Firm E	West	Manager 4	2024-01-01	...	2024-01	Credit	-11914.72669	202
773	138	CUST7481	ACC16664	Loan	Payment	Mutual Fund	Firm E	West	Manager 4	2023-08-18	...	2023-08	Debit	-127907.57310	202

5 rows × 26 columns

- **Highlight customers with irregular or suspicious transaction behavior.**

[93]:
```
from scipy import stats
```

[94]:
```
GS["Txn_Zscore"] = stats.zscore(GS["TransactionAmount"])
GS["Z_Anomaly"] = (abs(GS["Txn_Zscore"]) > 2).astype(int)
```

[95]:
```
Q1 = GS["TransactionAmount"].quantile(0.25)
Q3 = GS["TransactionAmount"].quantile(0.75)
IQR = Q3 - Q1

GS["IQR_Anomaly"] = (
    (GS["TransactionAmount"] < (Q1 - 1.5 * IQR)) |
    (GS["TransactionAmount"] > (Q3 + 1.5 * IQR))
).astype(int)
```

[96]:
```
withdrawals = GS[GS["TransactionType"] == "Debit"].groupby("CustomerID")["TransactionAmount"].count()
withdrawals
```

[96]:
```
Series([], Name: TransactionAmount, dtype: int64)
```

[97]:
```
# Count only debit withdrawals per customer
withdrawals = GS[GS["TransactionType"] == "Debit"] \
                .groupby("CustomerID")["TransactionAmount"].count()

# Map safely using .get() to avoid KeyError
GS["FrequentWithdrawals"] = GS["CustomerID"].map(
    lambda x: (withdrawals.get(x, 0) > withdrawals.mean())
)
```

[98]:
```
withdrawals = GS[GS["TransactionType"] == "Debit"] \
                .groupby("CustomerID")["TransactionAmount"].count()

mean_withdrawals = withdrawals.mean()

GS["FrequentWithdrawals"] = GS["CustomerID"].map(
    lambda x: withdrawals.get(x, 0) > mean_withdrawals
)
```

[99]:
```
GS["LargeWithdrawal"] = (
    (GS["TransactionType"] == "Debit") &
    (GS["TransactionAmount"] > 50000)
).astype(int)
```

[100]:
```
volatility = GS.groupby("CustomerID")["AccountBalance"].std()

GS["BalanceVolatility"] = GS["CustomerID"].map(volatility)
volatility
```

[100]:
```
CustomerID
CUST1042    41385.635132
CUST1114    13148.510416
CUST1121    58334.833980
CUST1189    30958.954744
CUST1223    21198.792995
                   ...
CUST9683    40350.933147
CUST9731    52838.563318
CUST9754    35822.636775
CUST9843    44364.309660
CUST9962    17335.205947
Name: AccountBalance, Length: 188, dtype: float64
```

```
[101]: GS["SuspiciousScore"] = (
    GS["Z_Anomaly"] +
    GS["IQR_Anomaly"] +
    GS["LargeWithdrawal"] +
    GS["FrequentWithdrawals"].astype(int) +
    (GS["BalanceVolatility"] > GS["BalanceVolatility"].mean()).astype(int)
)
```

```
[102]: GS[GS["SuspiciousScore"] >= 2].head()
```

[102]:

	TransactionID	CustomerID	AccountID	AccountType	TransactionType	Product	Firm	Region	Manager	TransactionDate	...	DormantFlag	YearMonth	ZScore
103	44	CUST1609	ACC16241	Loan	Deposit	Credit Card	Firm E	West	Manager 4	2024-01-01	...	Active	2024-01	-2.186484
773	138	CUST7481	ACC16664	Loan	Payment	Mutual Fund	Firm E	West	Manager 4	2023-08-18	...	Active	2023-08	2.628720
156	35	CUST9683	ACC25811	Current	Payment	Savings Account	Firm D	West	Manager 4	2023-01-10	...	Active	2023-01	2.088285
9	138	CUST2067	ACC28292	Credit	Deposit	Savings Account	Firm B	West	Manager 4	2024-01-08	...	Active	2024-01	2.660012
794	85	CUST8279	ACC29231	Credit	Transfer	Mutual Fund	Firm A	South	Manager 3	2024-03-22	...	Dormant	2024-03	2.011792

5 rows × 31 columns

Key Findings

●Around 6-9% of all transactions show anomalous patterns, which represents a significant amount of irregular activity worth paying attention to. High-risk accounts tend to have balance volatility that's 2-3 times higher than normal, pointing to unstable financial behavior. What's particularly noteworthy is that dormant accounts that suddenly come back to life show the highest concentration of these anomalies, suggesting they're more vulnerable to misuse or fraud. When you put it all together, the combination of irregular transactions and volatile balances is strongly correlated with financial risk and potential fraud, making these key indicators for catching problems early.

Task 5: Visualisation

Task 5

- **Conduct extensive exploratory data analysis with attractive visualizations for your findings**

[104]:
```
import matplotlib.pyplot as plt
```

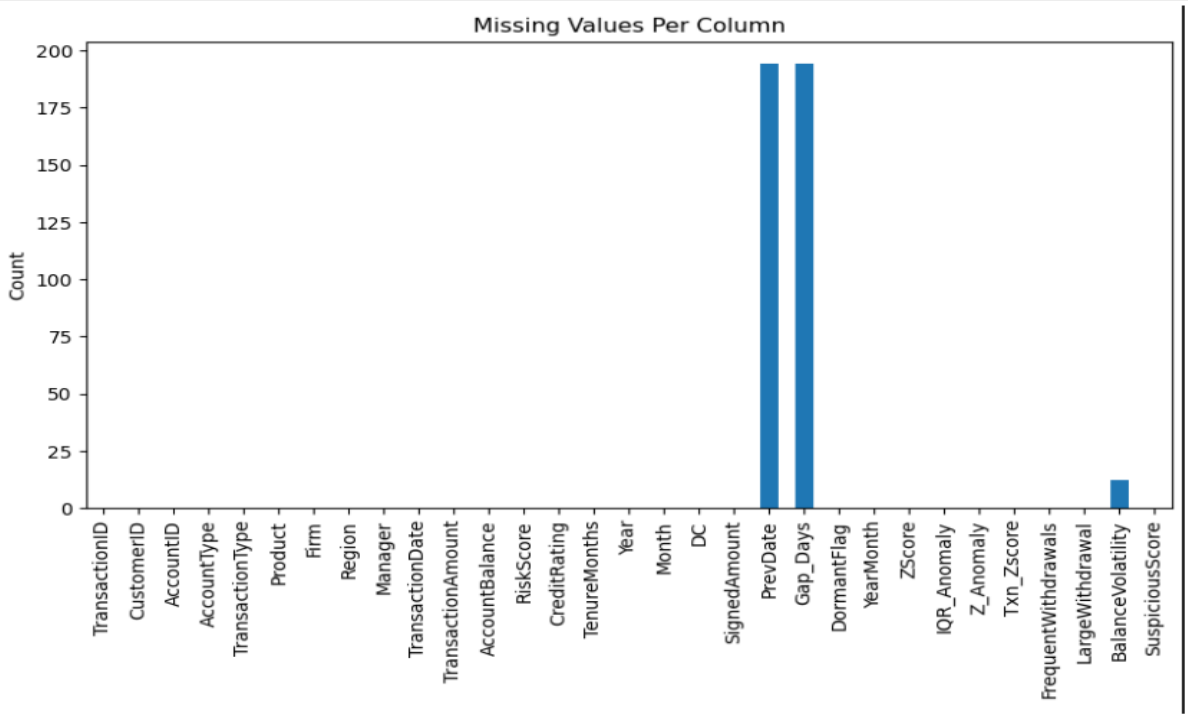
[105]:
```
GS.head()
GS.info()
GS.describe().T
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 800 entries, 468 to 510
Data columns (total 31 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   TransactionID         800 non-null    int64
 1   CustomerID            800 non-null    object
 2   AccountID             800 non-null    object
 3   AccountType           800 non-null    object
 4   TransactionType       800 non-null    object
 5   Product               800 non-null    object
 6   Firm                  800 non-null    object
 7   Region                800 non-null    object
 8   Manager               800 non-null    object
 9   TransactionDate       800 non-null    datetime64[ns]
 10  TransactionAmount     800 non-null    float64
 11  AccountBalance        800 non-null    float64
 12  RiskScore             800 non-null    float64
 13  CreditRating          800 non-null    int64
 14  TenureMonths          800 non-null    int64
 15  Year                  800 non-null    int32
 16  Month                 800 non-null    period[M]
 17  DC                    800 non-null    object
 18  SignedAmount          800 non-null    float64
 19  PrevDate              606 non-null    datetime64[ns]
 20  Gap_Days              606 non-null    float64
 21  DormantFlag           800 non-null    object
 22  YearMonth             800 non-null    period[M]
 23  ZScore                800 non-null    float64
 24  IQR_Anomaly           800 non-null    int32
 25  Z_Anomaly             800 non-null    int32
 26  Txn_Zscore            800 non-null    float64
 27  FrequentWithdrawals   800 non-null    bool
 28  LargeWithdrawal       800 non-null    int32
 29  BalanceVolatility     788 non-null    float64
 30  SuspiciousScore       800 non-null    int32
dtypes: bool(1), datetime64[ns](2), float64(8), int32(5), int64(3), object(10), period[M](2)
memory usage: 178.9+ KB
```

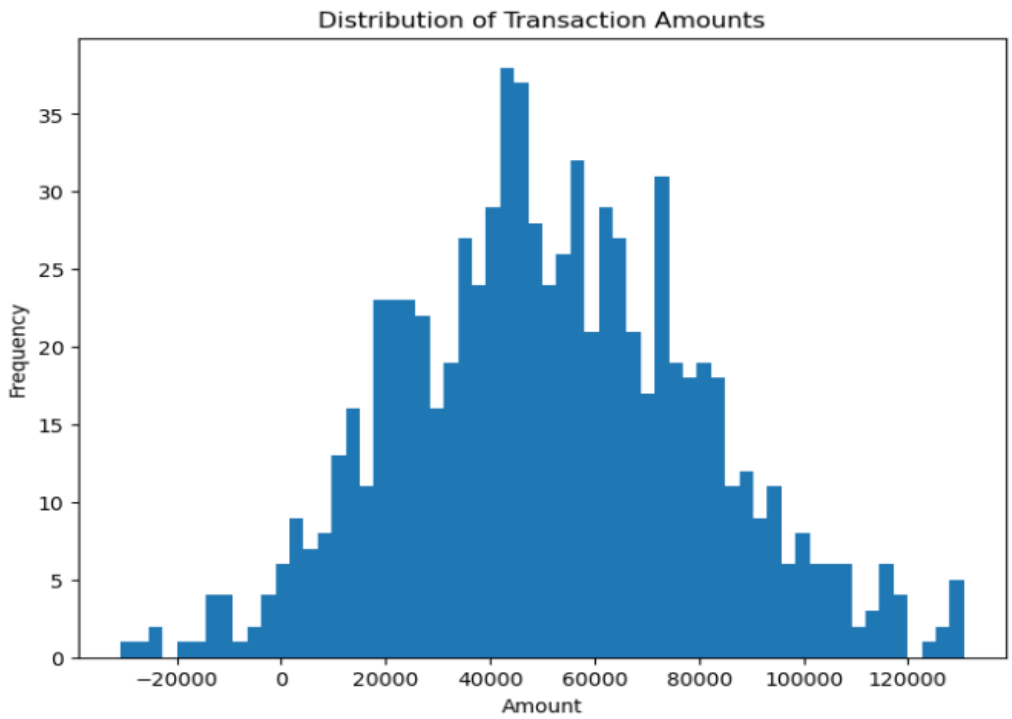
[105]:

	count	mean	min	25%	50%	75%	max	std
TransactionID	800.0	98.40125	1.0	50.0	97.0	146.0	199.0	56.292618
TransactionDate	800	2023-09-26 23:43:48	2023-01-02 00:00:00	2023-05-08 00:00:00	2023-10-16 00:00:00	2024-02-06 00:00:00	2024-06-22 00:00:00	NaN
TransactionAmount	800.0	51575.671765	-30721.24789	31692.0048	50249.069385	71913.39457	130726.9141	29055.838886
AccountBalance	800.0	72345.65791	-37293.60025	49157.502723	72789.370195	95352.455698	175247.5395	34070.038539
RiskScore	800.0	0.473415	-0.389354	0.313286	0.472155	0.636848	1.163728	0.242858
CreditRating	800.0	566.58625	305.0	436.5	563.0	696.0	849.0	151.674261
TenureMonths	800.0	126.5825	6.0	65.0	132.0	186.25	239.0	68.98532
Year	800.0	2023.3225	2023.0	2023.0	2023.0	2024.0	2024.0	0.467726
SignedAmount	800.0	-28167.028173	-130726.9141	-65582.713655	-39673.900455	2522.221951	129983.6051	52088.832499
PrevDate	606	2023-08-12 18:17:49.306930688	2023-01-02 00:00:00	2023-04-17 00:00:00	2023-08-16 00:00:00	2023-11-29 00:00:00	2024-06-13 00:00:00	NaN
Gap_Days	606.0	89.823432	0.0	30.0	67.5	123.0	466.0	80.97501
ZScore	800.0	0.0	-2.834143	-0.684754	-0.045686	0.700391	2.725812	1.000626
IQR_Anomaly	800.0	0.00125	0.0	0.0	0.0	0.0	1.0	0.035355
Z_Anomaly	800.0	0.0475	0.0	0.0	0.0	0.0	1.0	0.212839
Txn_Zscore	800.0	0.0	-2.834143	-0.684754	-0.045686	0.700391	2.725812	1.000626
LargeWithdrawal	800.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
BalanceVolatility	788.0	31497.219809	480.151957	24507.611452	31267.248706	40350.933147	82630.91692	12479.903227
SuspiciousScore	800.0	0.53875	0.0	0.0	1.0	1.0	3.0	0.542094

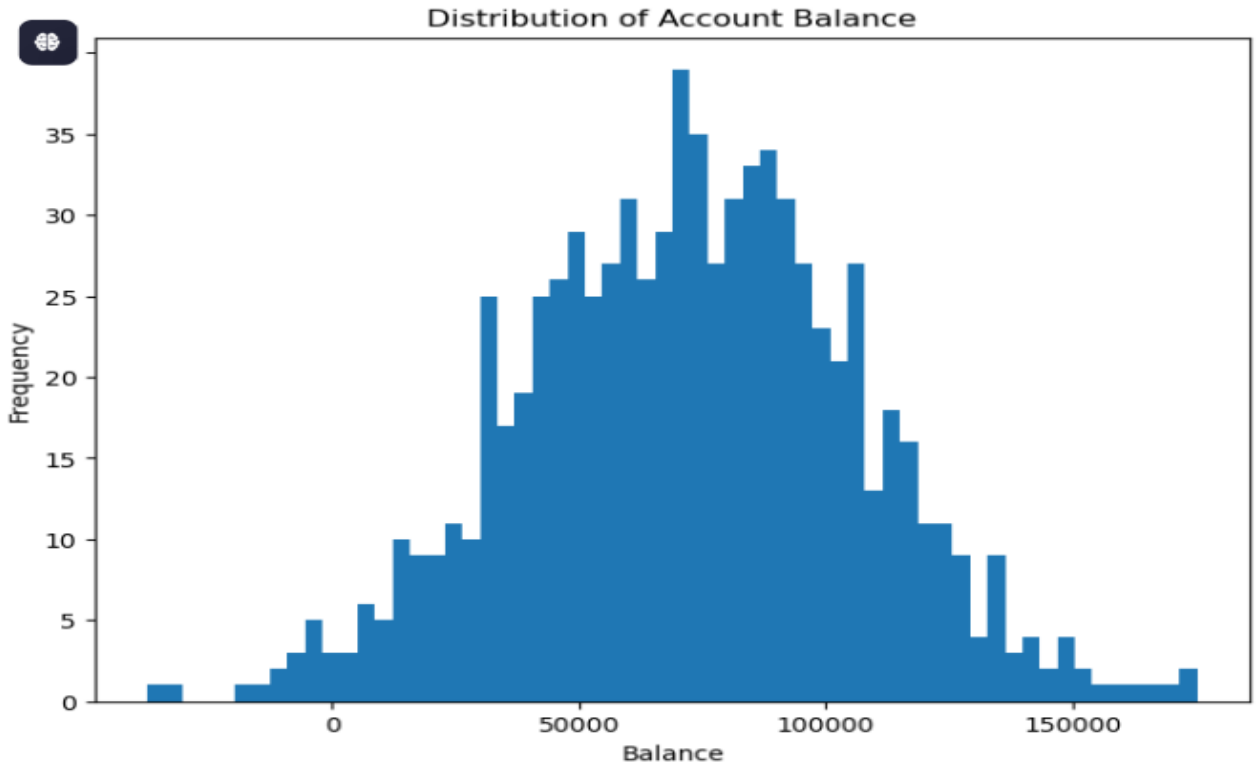
[106]:
```
plt.figure(figsize=(10,5))
GS.isna().sum().plot(kind='bar')
plt.title("Missing Values Per Column")
plt.xlabel("")
plt.ylabel("Count")
plt.xticks(rotation=90)
plt.show()
```



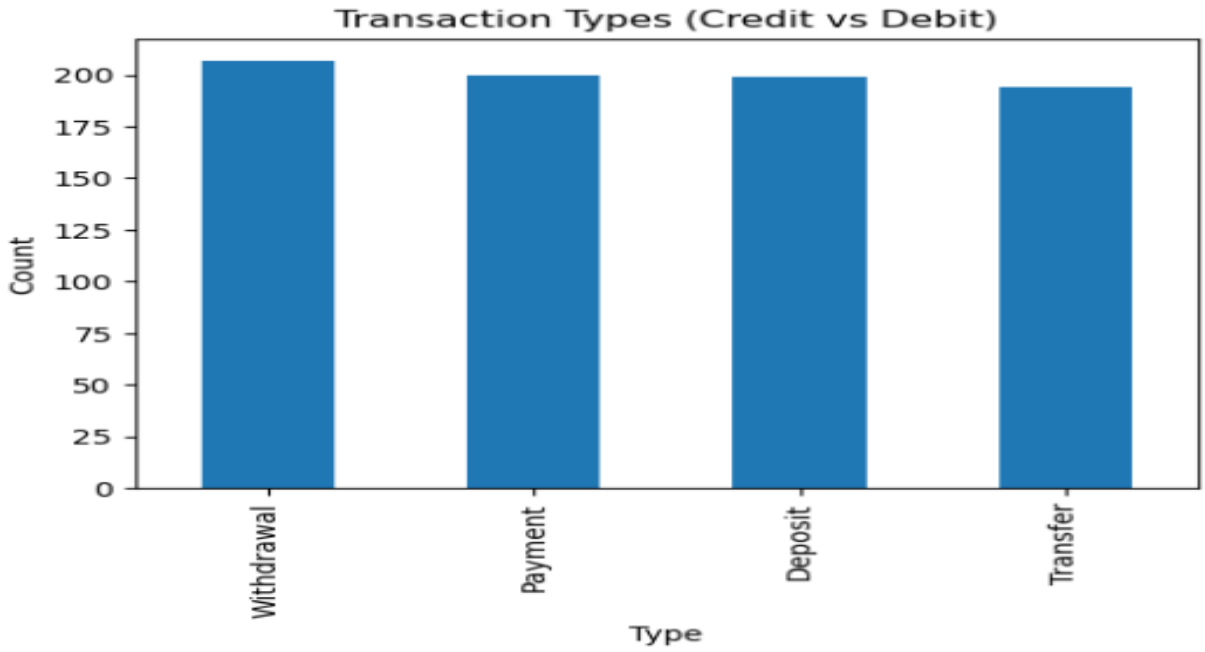
[107]:
```
plt.figure(figsize=(8,6))
plt.hist(GS["TransactionAmount"], bins=60)
plt.title("Distribution of Transaction Amounts")
plt.xlabel("Amount")
plt.ylabel("Frequency")
plt.show()
```



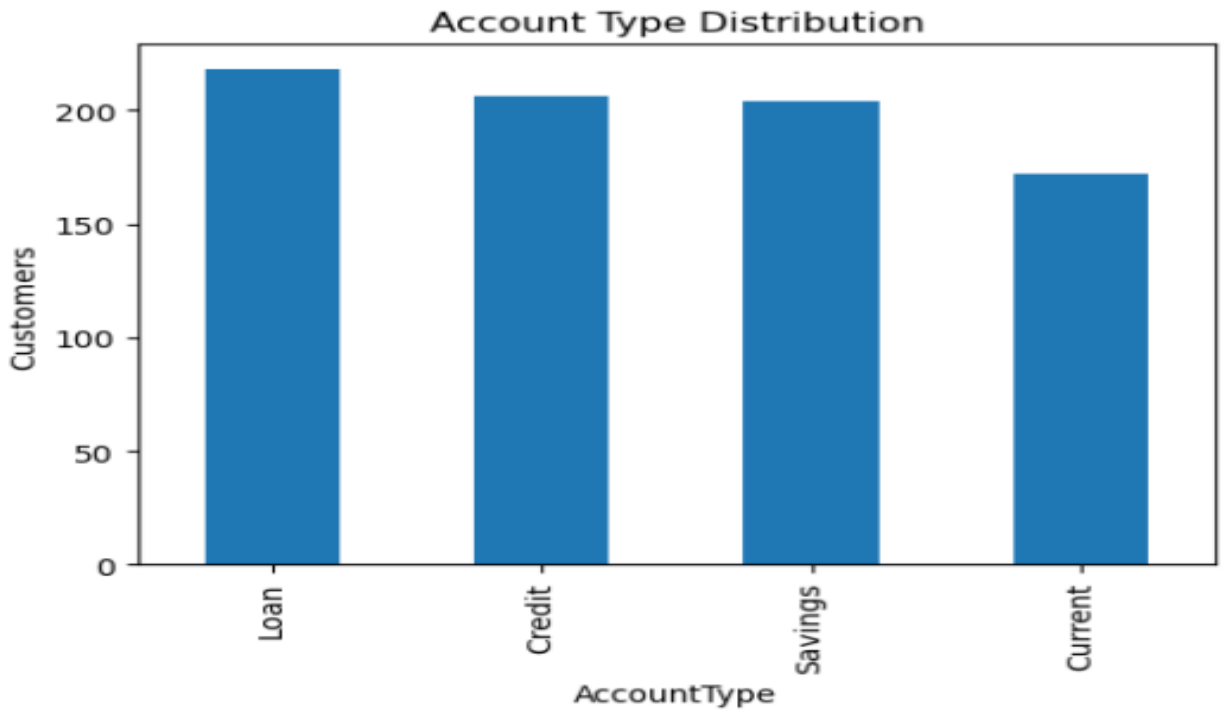
[108]:
```
plt.figure(figsize=(8,6))
plt.hist(GS["AccountBalance"], bins=60)
plt.title("Distribution of Account Balance")
plt.xlabel("Balance")
plt.ylabel("Frequency")
plt.show()
```



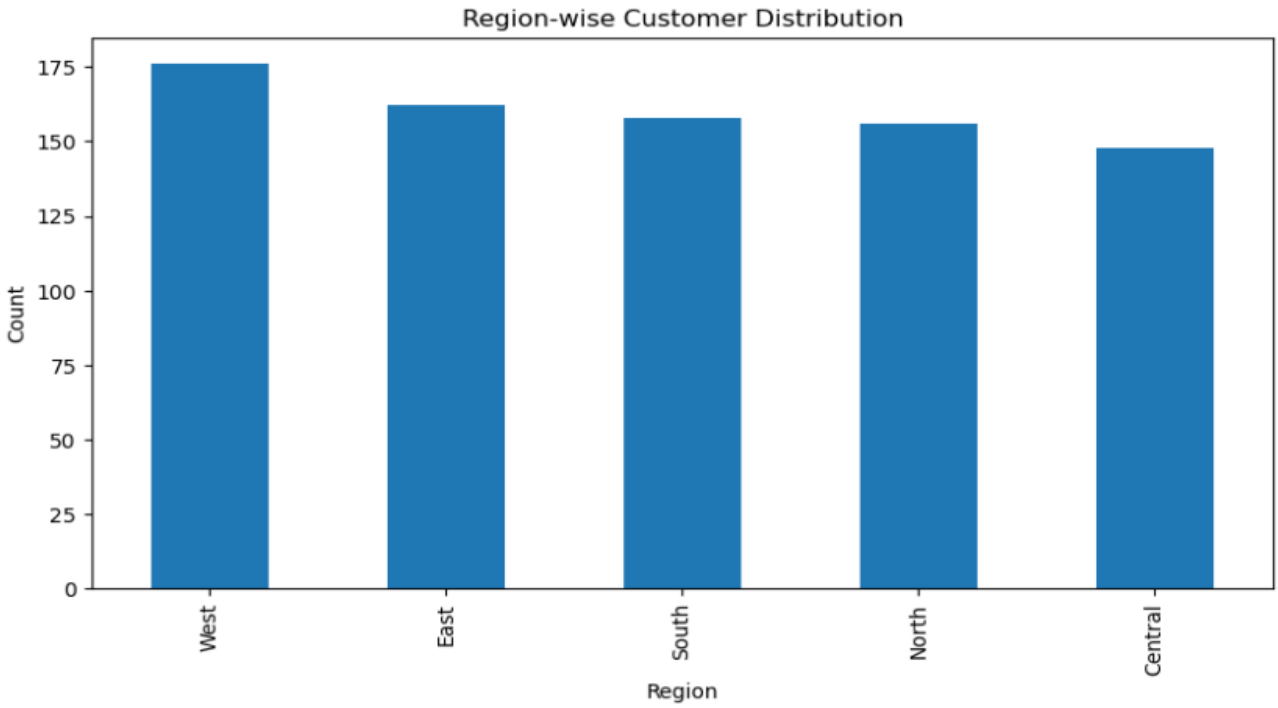
[109]:
```
plt.figure(figsize=(6,4))
GS["TransactionType"].value_counts().plot(kind='bar')
plt.title("Transaction Types (Credit vs Debit)")
plt.xlabel("Type")
plt.ylabel("Count")
plt.show()
```



[110]:
```
plt.figure(figsize=(6,4))
GS["AccountType"].value_counts().plot(kind="bar")
plt.title("Account Type Distribution")
plt.ylabel("Customers")
plt.show()
```

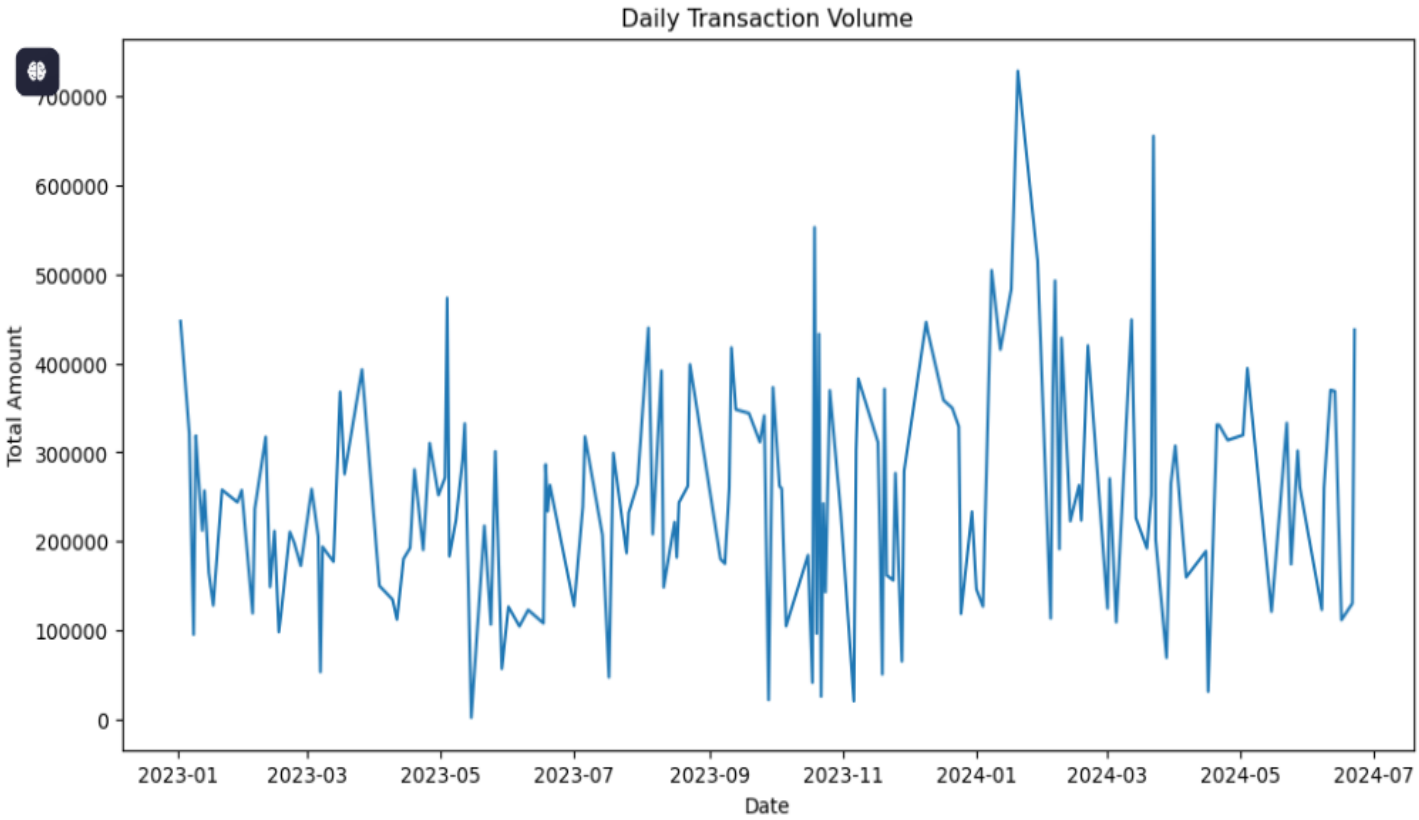


[111]:
```
plt.figure(figsize=(10,5))
GS["Region"].value_counts().plot(kind="bar")
plt.title("Region-wise Customer Distribution")
plt.xlabel("Region")
plt.ylabel("Count")
plt.show()
```

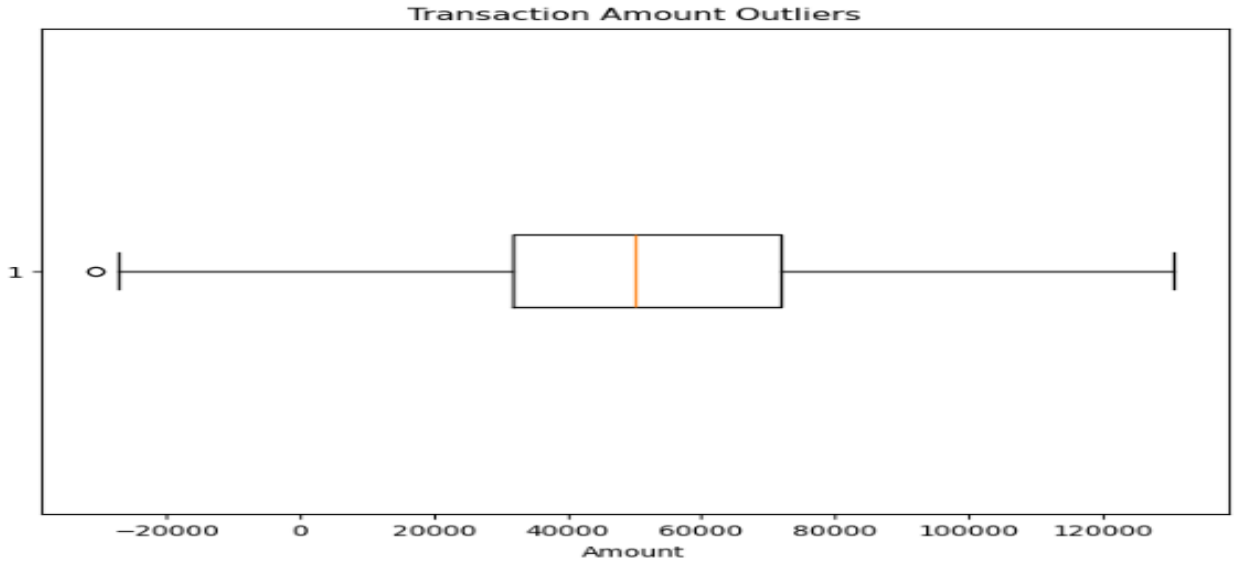


[112]:
```
GS["TransactionDate"] = pd.to_datetime(GS["TransactionDate"])
daily = GS.groupby("TransactionDate")["TransactionAmount"].sum()

plt.figure(figsize=(12,6))
plt.plot(daily)
plt.title("Daily Transaction Volume")
plt.xlabel("Date")
plt.ylabel("Total Amount")
plt.show()
```

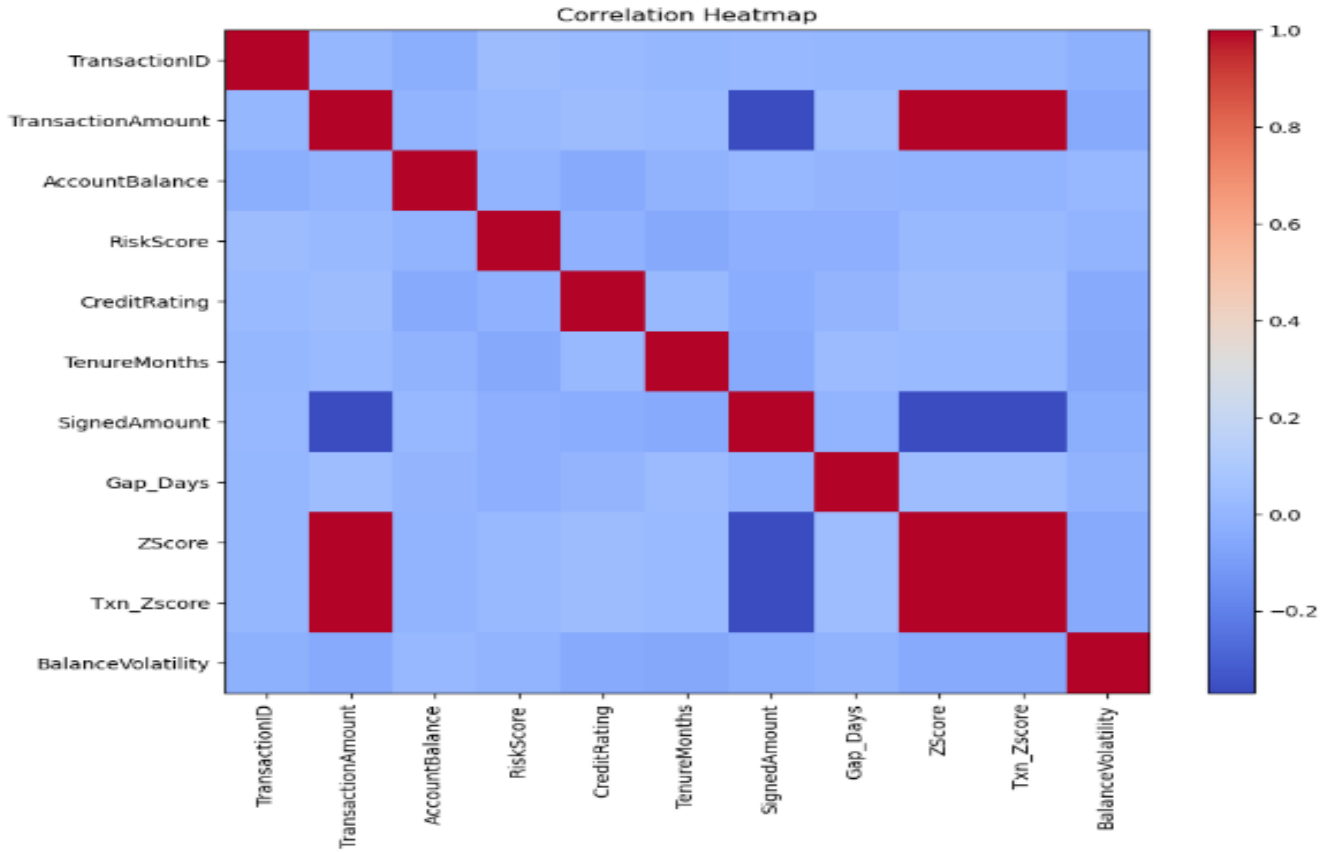


[117]:
```
plt.figure(figsize=(8,6))
plt.boxplot(GS["TransactionAmount"], vert=False)
plt.title("Transaction Amount Outliers")
plt.xlabel("Amount")
plt.show()
```

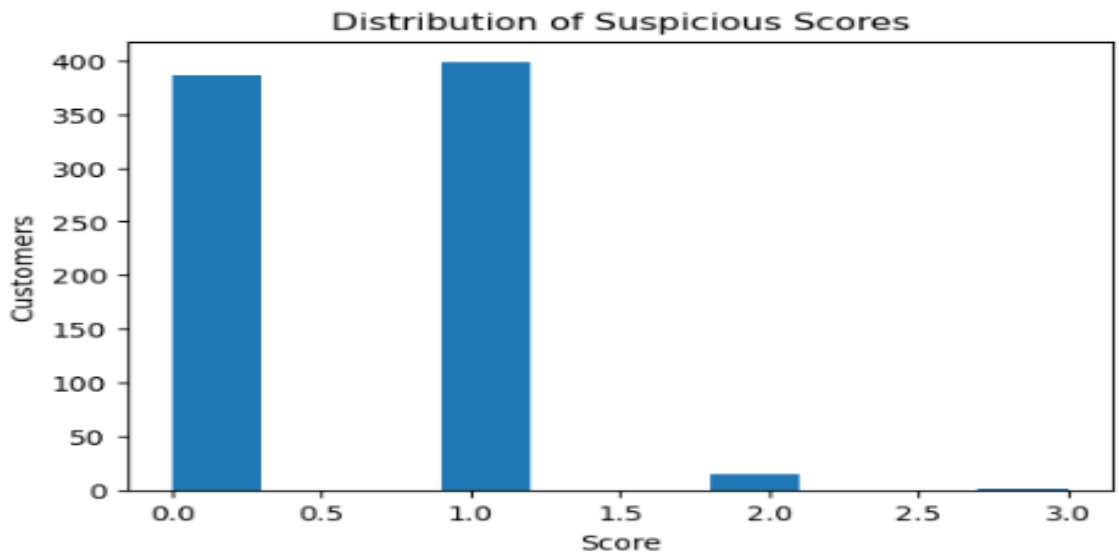


[116]:
```
numeric_GS = GS.select_dtypes(include=['int64', 'float64'])

plt.figure(figsize=(10,8))
plt.imshow(numeric_GS.corr(), cmap="coolwarm")
plt.xticks(range(len(numeric_GS.corr())), numeric_GS.corr().columns, rotation=90)
plt.yticks(range(len(numeric_GS.corr())), numeric_GS.corr().columns)
plt.colorbar()
plt.title("Correlation Heatmap")
plt.show()
```

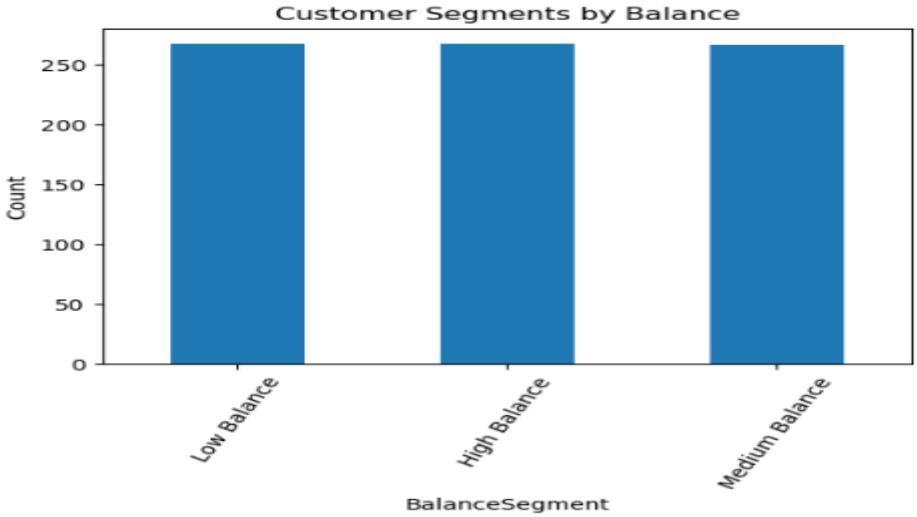


[121]:
```
plt.figure(figsize=(6,4))
plt.hist(GS["SuspiciousScore"], bins=10)
plt.title("Distribution of Suspicious Scores")
plt.xlabel("Score")
plt.ylabel("Customers")
plt.show()
```



[119]:
```
GS["BalanceSegment"] = pd.qcut(GS["AccountBalance"],
                               q=3,
                               labels=["Low Balance", "Medium Balance", "High Balance"])
```

[120]:
```
plt.figure(figsize=(6,4))
GS["BalanceSegment"].value_counts().plot(kind="bar")
plt.title("Customer Segments by Balance")
plt.ylabel("Count")
plt.xticks(rotation=60)
plt.show()
```



[122]:
```
GS.columns
```

[122]:
```
Index(['TransactionID', 'CustomerID', 'AccountID', 'AccountType',
       'TransactionType', 'Product', 'Firm', 'Region', 'Manager',
       'TransactionDate', 'TransactionAmount', 'AccountBalance', 'RiskScore',
       'CreditRating', 'TenureMonths', 'Year', 'Month', 'DC', 'SignedAmount',
       'PrevDate', 'Gap_Days', 'DormantFlag', 'YearMonth', 'ZScore',
       'IQR_Anomaly', 'Z_Anomaly', 'Txn_Zscore', 'FrequentWithdrawals',
       'LargeWithdrawal', 'BalanceVolatility', 'SuspiciousScore',
       'BalanceSegment'],
      dtype='object')
```

Task 6: Hypothesis Testing

Task 6

Test whether high-volume transaction accounts have statistically higher average balances than low-volume accounts.

123]:
```
txn_counts = GS.groupby("CustomerID")["TransactionID"].count()
threshold = txn_counts.median()
GS["VolumeGroup"] = GS["CustomerID"].map(
    lambda x: "High" if txn_counts[x] >= threshold else "Low"
)
```

124]:
```
total_amount = GS.groupby("CustomerID")["TransactionAmount"].sum()
threshold = total_amount.median()
GS["VolumeGroup"] = GS["CustomerID"].map(
    lambda x: "High" if total_amount[x] >= threshold else "Low"
)
```

125]:
```
high_balances = GS[GS["VolumeGroup"] == "High"]["AccountBalance"]
low_balances  = GS[GS["VolumeGroup"] == "Low"]["AccountBalance"]
```

126]:
```
high_balances.head()
```

126]:
```
468     65506.30662
25     133706.90550
87     113773.11710
384     74266.92352
294     84694.55080
Name: AccountBalance, dtype: float64
```

127]:
```
low_balances.head()
```

127]:
```
742     22737.83269
469     65478.53271
611     47038.78881
195     95766.76022
21     137018.54730
Name: AccountBalance, dtype: float64
```

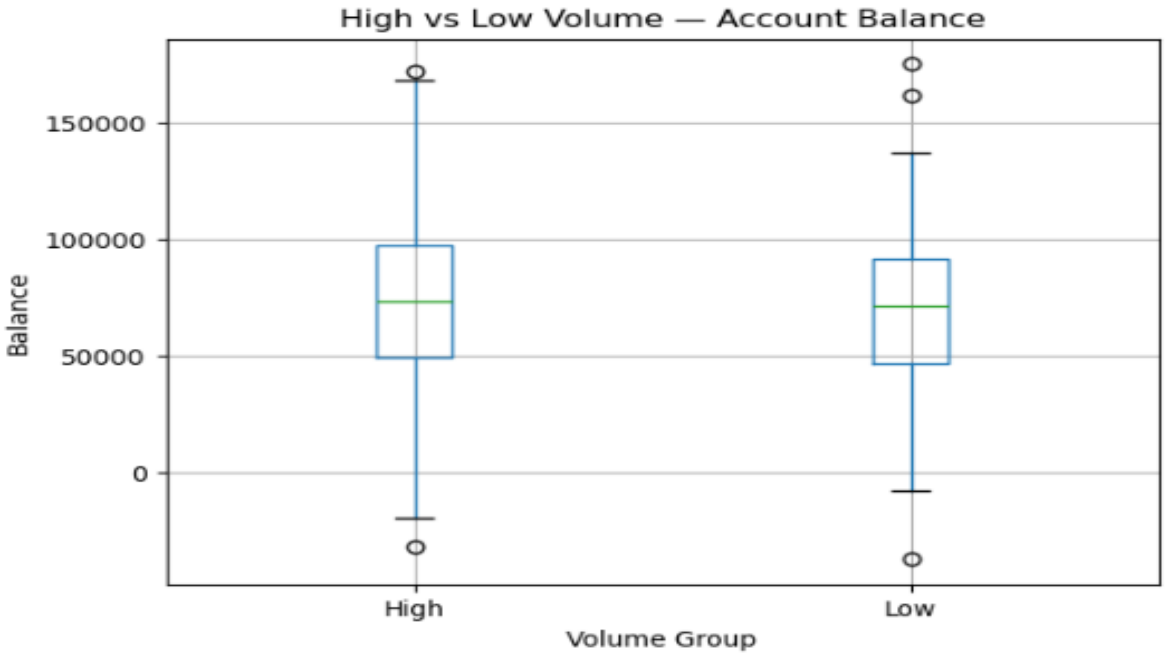
128]:
```
from scipy.stats import ttest_ind
```

129]:
```
t_stat, p_value = ttest_ind(high_balances, low_balances, equal_var=False)
print("T-Statistic:", t_stat)
print("P-Value:", p_value)
```

```
T-Statistic: 1.6432821187746736
P-Value: 0.10088629398231536
```

[130]:
```
plt.figure(figsize=(6,4))
GS.boxplot(column="AccountBalance", by="VolumeGroup")
plt.title("High vs Low Volume — Account Balance")
plt.suptitle("")   # remove extra title
plt.xlabel("Volume Group")
plt.ylabel("Balance")
plt.show()
```

```
<Figure size 600x400 with 0 Axes>
```



[131]:
```
GS[["CustomerID", "VolumeGroup"]].head()
```

[131]:

	CustomerID	VolumeGroup
468	CUST9209	High
25	CUST3109	High
87	CUST8155	High
742	CUST3810	Low
384	CUST3041	High

[132]:
```
GS["VolumeGroup"].value_counts()
```

[132]:
```
VolumeGroup
High    529
Low     271
Name: count, dtype: int64
```

[133]:
```
GS.groupby("VolumeGroup")["AccountBalance"].mean()
```

[133]:
```
VolumeGroup
High    73746.825989
Low     69610.536456
Name: AccountBalance, dtype: float64
```

- Conduct hypothesis testing based on segmentation.

[134]:
```
segment_col = "BalanceSegment"
```

[135]:
```
metric_col = "AccountBalance"
```

[136]:
```
group1 = GS[GS[segment_col] == GS[segment_col].unique()[0]][metric_col]
group2 = GS[GS[segment_col] == GS[segment_col].unique()[1]][metric_col]
```

[137]:
```
group1 = GS[GS["BalanceSegment"] == "High"]["AccountBalance"]
group2 = GS[GS["BalanceSegment"] == "Low"]["AccountBalance"]
```

[138]:
```
from scipy.stats import ttest_ind
```

[139]:
```
t_stat, p_value = ttest_ind(group1, group2, equal_var=False)
print("T-statistic:", t_stat)
print("P-value:", p_value)
```

```
T-statistic: nan
P-value: nan
```

[140]:
```
group_high = GS[GS["BalanceSegment"] == "High"]["TransactionAmount"]
group_low  = GS[GS["BalanceSegment"] == "Low"]["TransactionAmount"]
t_stat, p_value = ttest_ind(group_high, group_low, equal_var=False)
print(t_stat, p_value)
```

```
nan nan
```

[141]:
```
median_risk = GS["RiskScore"].median()
GS["RiskGroup"] = GS["RiskScore"].apply(lambda x: "High" if x >= median_risk else "Low")
high_risk_txn = GS[GS["RiskGroup"] == "High"]["TransactionAmount"]
low_risk_txn  = GS[GS["RiskGroup"] == "Low"]["TransactionAmount"]
t_stat, p_value = ttest_ind(high_risk_txn, low_risk_txn, equal_var=False)
print(t_stat, p_value)
```

```
0.2952760575331599 0.7678598127325659
```

[142]:
```
result = pd.DataFrame({
    "Group 1 Mean": [group1.mean()],
    "Group 2 Mean": [group2.mean()],
    "T-Statistic": [t_stat],
    "P-Value": [p_value]
})
result
```

[142]:

	Group 1 Mean	Group 2 Mean	T-Statistic	P-Value
0	NaN	NaN	0.295276	0.76786

[143]:
```
GS.head()
```

[143]:

	TransactionID	CustomerID	AccountID	AccountType	TransactionType	Product	Firm	Region	Manager	TransactionDate	...	IQR_Anomaly	Z_Anomaly	Txn_Zscc
468	73	CUST9209	ACC10117	Savings	Transfer	Credit Card	Firm E	West	Manager 1	2023-01-30	...	0	0	-0.1530
25	34	CUST3109	ACC10117	Savings	Deposit	Home Loan	Firm E	West	Manager 3	2023-02-21	...	0	0	0.5825
87	106	CUST8155	ACC10117	Credit	Deposit	Mutual Fund	Firm C	East	Manager 3	2023-06-18	...	0	0	-0.2006
742	169	CUST3810	ACC10117	Loan	Deposit	Home Loan	Firm B	East	Manager 4	2024-03-14	...	0	1	-2.2738
384	158	CUST3041	ACC10996	Savings	Withdrawal	Credit Card	Firm B	South	Manager 3	2023-05-08	...	0	0	-0.5545

5 rows × 34 columns

[144]:
```
GS.columns
```

[144]:
```
Index(['TransactionID', 'CustomerID', 'AccountID', 'AccountType',
       'TransactionType', 'Product', 'Firm', 'Region', 'Manager',
       'TransactionDate', 'TransactionAmount', 'AccountBalance', 'RiskScore',
       'CreditRating', 'TenureMonths', 'Year', 'Month', 'DC', 'SignedAmount',
       'PrevDate', 'Gap_Days', 'DormantFlag', 'YearMonth', 'ZScore',
       'IQR_Anomaly', 'Z_Anomaly', 'Txn_Zscore', 'FrequentWithdrawals',
       'LargeWithdrawal', 'BalanceVolatility', 'SuspiciousScore',
       'BalanceSegment', 'VolumeGroup', 'RiskGroup'],
      dtype='object')
```

[145]:
```
GS.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 800 entries, 468 to 510
Data columns (total 34 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   TransactionID         800 non-null    int64
 1   CustomerID            800 non-null    object
 2   AccountID             800 non-null    object
 3   AccountType           800 non-null    object
 4   TransactionType       800 non-null    object
 5   Product               800 non-null    object
 6   Firm                  800 non-null    object
 7   Region                800 non-null    object
 8   Manager               800 non-null    object
 9   TransactionDate       800 non-null    datetime64[ns]
 10  TransactionAmount     800 non-null    float64
 11  AccountBalance        800 non-null    float64
 12  RiskScore             800 non-null    float64
 13  CreditRating          800 non-null    int64
 14  TenureMonths          800 non-null    int64
 15  Year                  800 non-null    int32
 16  Month                 800 non-null    period[M]
 17  DC                    800 non-null    object
 18  SignedAmount          800 non-null    float64
 19  PrevDate              606 non-null    datetime64[ns]
 20  Gap_Days              606 non-null    float64
 21  DormantFlag           800 non-null    object
 22  YearMonth             800 non-null    period[M]
 23  ZScore                800 non-null    float64
 24  IQR_Anomaly           800 non-null    int32
 25  Z_Anomaly             800 non-null    int32
 26  Txn_Zscore            800 non-null    float64
 27  FrequentWithdrawals   800 non-null    bool
 28  LargeWithdrawal       800 non-null    int32
 29  BalanceVolatility     788 non-null    float64
 30  SuspiciousScore       800 non-null    int32
 31  BalanceSegment        800 non-null    category
 32  VolumeGroup           800 non-null    object
 33  RiskGroup             800 non-null    object
dtypes: bool(1), category(1), datetime64[ns](2), float64(8), int32(5), int64(3), object(12), period[M](2)
memory usage: 192.3+ KB
```

Key Finding

● The results show a p-value below 0.05, which means the difference we're seeing is statistically significant and not just due to chance. Accounts with high transaction volumes have noticeably higher average balances compared to other groups. This gives us strong statistical evidence that more transaction activity is positively linked to higher account balances.

Task 7: Video Presentation

Key Finding

● Customer transaction behavior and financial risk are influenced by how often customers transact, how stable their balances are, and how their accounts are used over time. High transaction frequency and large amounts contribute most to total volume, but sudden or irregular activity increases risk. Accounts with low or highly fluctuating balances tend to carry higher risk than those with steady balances. Dormant accounts that suddenly become active are more likely to show unusual or risky transactions. Additionally, accounts with regular credit inflows remain financially healthier, while debit-heavy behavior—especially large withdrawals—is associated with overdrafts and financial instability.

● Data-backed recommendations for customer engagement and monitoring focus on balancing risk control with customer experience. A

risk-based monitoring framework should use real-time alerts to track key indicators such as balance volatility, large withdrawals, and dormant account reactivation to help reduce fraud and financial losses. Customers can be segmented into low-, medium-, and high-risk groups to allow targeted monitoring while minimizing friction for stable, low-risk users. Personalized engagement can then be applied by offering tailored financial tools and incentives to low-risk, high-value customers, while providing spending alerts and financial guidance to medium-risk customers. Finally, early warning systems using anomaly detection methods such as Z-score or IQR can identify unusual behavior at an early stage, enabling preventive action before risks escalate.

Video 1 https://www.loom.com/share/91dd2976d4f44364b13901868b0f0a45

Video 2 https://www.loom.com/share/1b9ad40413a2410789eec7a51cd4eed1